

Ad Soyad: İlayda ÖZTÜRK

Öğrenci No: 23080410302

Ders: Programlama Dilleri

Öğretim Görevlisi: Dr. Öğr. Üyesi Songül KARAKUŞ – Arş. Gör. NİLAY BOZKURT

KELİME FREKANSI ANALİZİ RAPORU

1. GİRİŞ

Programlama dilleri, bilgisayar sistemlerine belirli görevleri yerine getirmesi için komut vermeyi sağlayan temel araçlardır. Günümüzde farklı ihtiyaçlara yönelik geliştirilmiş çok sayıda programlama dili bulunmaktadır. Bu diller; performans, kullanım kolaylığı, okunabilirlik, geliştirme süresi ve programlama paradigması açısından birbirlerinden ayrılmaktadır.

Bazı programlama dilleri hızlı geliştirme süreçleri sunarken bazıları performans açısından daha güçlü çözümler üretmektedir. Bunun yanında bazı diller nesne yönelimli programlama yaklaşımını benimserken bazıları prosedürel ya da betik tabanlı yaklaşımlar kullanmaktadır. Bu nedenle aynı problemin farklı programlama dilleri kullanılarak çözülmesi, diller arasındaki farkların daha iyi anlaşılmasını sağlamaktadır.

Bu çalışmada verilen bir metin üzerinde temel metin işleme işlemleri gerçekleştirilmiştir. Çalışmanın amacı; farklı programlama dillerinin aynı problem karşısındaki çözüm yöntemlerini incelemek ve bu dilleri çeşitli açılardan karşılaştırmaktır.

Gerçekleştirilen uygulamada:

- Metin içerisindeki kelimeler ayrılmış,
- Her kelimenin kaç kez tekrar ettiği hesaplanmış,
- Kelime frekansları belirlenmiş,
- En çok kullanılan ilk beş kelime ekrana yazdırılmıştır.

Çalışma kapsamında üç farklı programlama dili kullanılmıştır:

- Python
- Java
- C

Bu diller farklı programlama paradigmasını temsil etmektedir. Python dili betik dili yaklaşımını temsil ederken, Java nesne yönelimli programlama yaklaşımını temsil etmektedir. C dili ise prosedürel ve düşük seviyeli programlama yaklaşımının önemli örneklerinden biridir.

Bu çalışma sayesinde aynı problemin farklı dillerde nasıl çözüldüğü, kullanılan veri yapıları, kod uzunluğu, okunabilirlik, performans ve geliştirme süreci açısından değerlendirilmiştir.

2. PROBLEM TANIMI

Bu çalışmada çözülmesi istenen problem, verilen bir metin üzerinde kelime frekans analizi gerçekleştirilmesidir.

Kelime frekans analizi, bir metin içerisinde geçen kelimelerin tekrar sayılarını belirlemeyi amaçlayan temel bir metin işleme yöntemidir. Bu yöntem özellikle doğal dil işleme, veri analizi, arama motorları, sosyal medya analizleri ve içerik sınıflandırma sistemlerinde yaygın olarak kullanılmaktadır.

Problemın çözümünde aşağıdaki işlemler gerçekleştirilmiştir:

1. Verilen metin içerisindeki kelimelerin ayrılması
2. Büyük ve küçük harf farklılıklarının ortadan kaldırılması
3. Her kelimenin tekrar sayısının hesaplanması
4. Kelimelerin frekanslarına göre sıralanması
5. En çok tekrar eden ilk beş kelimenin ekrana yazdırılması

Bu işlemler gerçekleştirilirken her programlama dilinin kendi veri yapıları ve kütüphaneleri kullanılmıştır.

Çalışma sırasında kullanılan örnek metin aşağıdaki gibidir:

“Bu bir örnek metin. Bu metin içinde bazı kelimeler tekrar ediyor. Kelimeler tekrar ettikçe bu kelimeler sayılacak.”

Bu metin üzerinde gerçekleştirilen analiz sonucunda hangi kelimelerin daha sık kullanıldığı belirlenmiştir.

3. KULLANILAN PROGRAMLAMA DİLLERİ

Bu çalışmada üç farklı programlama dili tercih edilmiştir. Diller seçilirken farklı programlama paradigmasını temsil etmelerine dikkat edilmiştir.

Tablo 1. Seçilen Programlama Dilleri ve Türleri.

Programlama Dili	Türü
Python	Betik Dili
Java	Nesne Yönelimli
C	Prosedürel / Düşük Seviyeli

3.1 Python

Python, yüksek seviyeli, yorumlanan ve okunabilirliği ön planda tutularak geliştirilmiş modern bir programlama dilidir. Basit ve anlaşılır sözdizimi sayesinde öğrenmesi ve kullanılması kolaydır. Bu nedenle günümüzde hem eğitim amaçlı hem de profesyonel yazılım geliştirme süreçlerinde oldukça yaygın şekilde kullanılmaktadır.

Python dili özellikle:

- Veri analizi
- Yapay zeka
- Makine öğrenmesi
- Web geliştirme
- Siber güvenlik
- Otomasyon sistemleri
- Bilimsel hesaplamalar

gibi birçok alanda aktif olarak tercih edilmektedir.

Python'un en önemli avantajlarından biri geniş standart kütüphane desteğine sahip olmasıdır. Geliştiriciler birçok işlemi ek kod yazmadan hazır modüller yardımıyla gerçekleştirebilmektedir. Bu durum geliştirme sürecini hızlandırmakta ve kod karmaşıklığını azaltmaktadır.

Bu çalışmada Python dilinin re ve collections kütüphaneleri kullanılmıştır. re kütüphanesi sayesinde regex yapıları kullanılarak metin içerisindeki kelimeler kolayca ayrıştırılmıştır. collections kütüphanesi içerisinde bulunan Counter veri yapısı ise kelimelerin tekrar sayılarını otomatik olarak hesaplamıştır.

Python dilinin kısa ve anlaşılır sözdizimine sahip olması sayesinde aynı problem diğer dillere göre daha az kod ile çözülebilmektedir. Bu durum hem okunabilirliği artırmış hem de hata yapma ihtimalini azaltmıştır.

Ayrıca Python dili geliştiricilere hızlı prototipleme imkanı sunmaktadır. Özellikle küçük ve orta ölçekli projelerde geliştirme süreci oldukça hızlı ilerlemektedir. Bu nedenle Python dili günümüzde en popüler programlama dillerinden biri haline gelmiştir.

3.2 Java

Java, nesne yönelimli programlama yaklaşımını temel alan güçlü ve yaygın olarak kullanılan bir programlama dilidir. 1995 yılında Sun Microsystems tarafından geliştirilmiş olup günümüzde Oracle tarafından desteklenmektedir. Güvenilirliği, platform bağımsız çalışabilmesi ve güçlü standart kütüphanelere sahip olması nedeniyle büyük ölçekli yazılım projelerinde yaygın olarak tercih edilmektedir.

Java dilinin en önemli özelliklerinden biri “Write Once, Run Anywhere” yaklaşımıdır. Bu yaklaşım sayesinde Java ile geliştirilen uygulamalar, Java Virtual Machine (JVM) bulunan farklı işletim sistemlerinde yeniden derlenmeden çalıştırılabilmektedir. Bu durum Java'yı platform bağımsız hale getirmektedir.

Java dili özellikle:

- Kurumsal yazılım geliştirme
- Mobil uygulama geliştirme
- Web tabanlı sistemler
- Finans ve bankacılık uygulamaları
- Masaüstü uygulamaları
- Büyük veri sistemleri

gibi alanlarda yaygın olarak kullanılmaktadır.

Bu çalışmada Java dilinin güçlü veri yapıları ve standart kütüphanelerinden yararlanılmıştır. Kelime frekanslarını tutmak amacıyla HashMap veri yapısı kullanılmıştır. Metin içerisindeki kelimelerin ayrıştırılması için Pattern ve Matcher sınıflarından yararlanılmıştır. Ayrıca kelimelerin frekans değerlerine göre sıralanması amacıyla modern Java yapılarından biri olan Stream API kullanılmıştır.

Java dili Python'a göre daha fazla sözdizimi gerektirmektedir. Ancak bu durum kodun daha düzenli, kontrollü ve sürdürülebilir olmasını sağlamaktadır. Özellikle büyük ölçekli projelerde nesne yönelimli yaklaşım sayesinde kod yönetimi daha kolay hale gelmektedir.

Bunun yanında Java dili güçlü hata kontrol mekanizmalarına sahiptir. Derleme aşamasında birçok hata tespit edilebildiği için yazılım güvenilirliği artmaktadır.

Bu çalışmada Java dili kullanılarak gerçekleştirilen çözüm, veri yapıları ve nesne yönelimli yaklaşım sayesinde düzenli ve okunabilir bir yapı sunmuştur.

3.3 C

C dili, düşük seviyeli ve prosedürel programlama yaklaşımını temel alan güçlü ve köklü bir programlama dilidir. 1970'li yıllarda Dennis Ritchie tarafından geliştirilmiş olup günümüzde halen en yaygın kullanılan programlama dillerinden biri olmayı sürdürmektedir. Özellikle donanımına yakın çalışabilmesi ve yüksek performans sunabilmesi nedeniyle sistem programlama alanında büyük öneme sahiptir.

C dili günümüzde özellikle:

- İşletim sistemleri
- Gömülü sistemler
- Sürücü yazılımları
- Mikrodenetleyici uygulamaları
- Sistem yazılımları
- Oyun motorları
- Performans gerektiren uygulamalar

gibi alanlarda yaygın olarak kullanılmaktadır.

C dilinin en önemli özelliklerinden biri programcıya bellek yönetimi ve veri işlemleri üzerinde doğrudan kontrol sağlamasıdır. Bu durum performans açısından önemli avantajlar sunsa da geliştirme sürecini daha karmaşık hale getirmektedir.

Diğer modern programlama dillerinde bulunan birçok hazır veri yapısı ve kütüphane C dilinde doğrudan bulunmamaktadır. Bu nedenle string işlemleri, veri saklama işlemleri ve sıralama algoritmaları çoğu zaman manuel olarak geliştirilmektedir.

Bu çalışmada C dili kullanılarak gerçekleştirilen çözümde kelimeler strtok() fonksiyonu yardımıyla ayrılmıştır. Her kelimenin tekrar sayısını saklamak amacıyla struct yapısı kullanılmıştır. Frekans hesaplama işlemleri karşılaştırmalar yapılarak manuel şekilde gerçekleştirilmiştir. Ayrıca kelimelerin tekrar sayılarına göre sıralanması için Bubble Sort algoritmasından yararlanılmıştır.

C dili diğer dillere göre daha fazla kod yazılmasını gerektirmiştir. Özellikle string işlemleri ve veri yönetimi süreçleri daha detaylı şekilde ele alınmıştır. Bununla birlikte düşük seviyeli yapısı sayesinde performans açısından oldukça güçlü sonuçlar sunmaktadır.

Bu çalışmada C dili kullanılarak geliştirilen çözüm, programcıya veri yönetimi üzerinde daha fazla kontrol sağlarken aynı zamanda prosedürel programlama yaklaşımının çalışma mantığını daha açık şekilde göstermiştir.

4. PYTHON İLE GERÇEKLEŞTİRİLEN ÇÖZÜM

Python dili kullanılarak gerçekleştirilen çözümde metin işleme işlemleri için re ve collections kütüphanelerinden yararlanılmıştır. Python dilinin güçlü standart kütüphane desteği sayesinde kelime frekans analizi işlemleri daha kısa ve okunabilir bir şekilde gerçekleştirilebilmiştir.

İlk aşamada kullanıcı tarafından verilen metin küçük harfe dönüştürülmüştür. Bu işlemin yapılmasının temel nedeni büyük ve küçük harf farklılıklarını ortadan kaldırmaktır. Örneğin “Bu” ve “bu” kelimelerinin aynı kelime olarak değerlendirilmesi sağlanmıştır.

Daha sonra re kütüphanesi kullanılarak regex (regular expression) yapısı ile metin içerisindeki kelimeler ayrıştırılmıştır. Regex yapısı sayesinde noktalama işaretleri temizlenmiş ve yalnızca kelimeler elde edilmiştir. Bu yöntem metin işleme uygulamalarında oldukça yaygın şekilde kullanılmaktadır.

Kelimeler ayrıştırıldıktan sonra collections kütüphanesi içerisinde bulunan Counter veri yapısı kullanılmıştır. Counter, listedeki elemanların kaç kez tekrar ettiğini otomatik olarak hesaplayan güçlü bir veri yapısıdır. Bu yapı sayesinde kelime frekanslarının hesaplanması oldukça kolay hale gelmiştir.

Frekans hesaplama işlemlerinin ardından kelimeler tekrar sayılarına göre sıralanmış ve en çok kullanılan ilk beş kelime ekrana yazdırılmıştır.

Python çözümünde gerçekleştirilen işlemler aşağıdaki gibidir:

- Metni küçük harfe dönüştürme
- Regex kullanarak kelimeleri ayırma
- Kelime frekanslarını hesaplama
- Kelimeleri frekans değerlerine göre sıralama

- En çok kullanılan ilk beş kelimeyi ekrana yazdırma

Python çözümünün en önemli avantajı kısa ve anlaşılır kod yapısına sahip olmasıdır. Hazır kütüphaneler sayesinde karmaşık işlemler oldukça az kod kullanılarak gerçekleştirilebilmiştir. Bu durum hem geliştirme sürecini hızlandırmış hem de kodun okunabilirliğini artırmıştır.

Ayrıca Python dili hata ayıklama ve geliştirme süreçlerinde de önemli kolaylıklar sağlamaktadır. Bu nedenle veri analizi ve metin işleme uygulamalarında oldukça sık tercih edilmektedir.

Python kaynak kodu aşağıda verilmiştir:

```
python > ...
1 import re
2 from collections import Counter
3
4 def top_5_words_python(text):
5     # Kelimeleri ayır
6     words = re.findall(r'\b\w+\b', text.lower())
7
8     # Frekans hesabı
9     word_counts = Counter(words)
10
11     # Tüm kelime frekansları
12     print("--- Tüm Kelime Frekansları ---")
13     for word, count in word_counts.items():
14         print(f"{word}: {count}")
15
16     # İlk 5 kelime
17     print("\n--- En Çok Geçen İlk 5 Kelime ---")
18     for word, count in word_counts.most_common(5):
19         print(f"{word}: {count}")
20
21 sample_text = """
22 Bu bir örnek metin. Bu metin içinde bazı kelimeler tekrar ediyor.
23 Kelimeler tekrar ettikçe bu kelimeler sayılacak.
24 """
25
26 top_5_words_python(sample_text)
```

Şekil 1: Python Kaynak Kodu

Python programının çalıştırılması sonucunda elde edilen çıktı aşağıda gösterilmiştir.

```
--- Tüm Kelime Frekansları ---
bu: 3
bir: 1
örnek: 1
metin: 2
içinde: 1
bazı: 1
kelimeler: 3
tekrar: 2
ediyor: 1
ettikçe: 1
sayılacak: 1

--- En Çok Geçen İlk 5 Kelime ---
bu: 3
kelimeler: 3
metin: 2
tekrar: 2
bir: 1
```

Şekil 2: Python Program Çıktısı

Ayrıca Python dilinde geliştirme süreci diğer dillere göre daha hızlı ilerlemiştir.

5. JAVA İLE GERÇEKLEŞTİRİLEN ÇÖZÜM

Java dili kullanılarak gerçekleştirilen çözümde nesne yönelimli programlama yaklaşımı temel alınmıştır. Java'nın güçlü standart kütüphaneleri ve veri yapıları sayesinde kelime frekans analizi işlemleri düzenli ve kontrollü bir şekilde gerçekleştirilmiştir.

İlk aşamada metin küçük harfe dönüştürülmüştür. Bu işlem sayesinde büyük ve küçük harf farklılıklarının analiz sonucunu etkilemesi engellenmiştir. Böylece aynı kelimenin farklı yazımları tek bir kelime olarak değerlendirilmiştir.

Metin içerisindeki kelimelerin ayrıştırılması için Pattern ve Matcher sınıfları kullanılmıştır. Bu sınıflar Java'nın regex desteğini sağlamaktadır. Regex yapısı sayesinde noktalama işaretleri temizlenmiş ve yalnızca kelimeler elde edilmiştir.

Kelime frekanslarını saklamak amacıyla HashMap veri yapısından yararlanılmıştır. HashMap, anahtar-değer mantığıyla çalışan güçlü bir veri yapısıdır. Bu çalışmada kelimeler anahtar (key), tekrar sayıları ise değer (value) olarak tutulmuştur. Her yeni kelime bulunduğu ilgili kelimenin tekrar sayısı artırılmıştır.

Frekans hesaplama işlemleri tamamlandıktan sonra Stream API kullanılarak kelimeler tekrar sayılarına göre sıralanmıştır. Stream API, Java'nın modern veri işleme yapılarından biridir ve veriler üzerinde daha okunabilir ve düzenli işlemler yapılmasını sağlamaktadır.

Java çözümünde gerçekleştirilen işlemler aşağıdaki gibidir:

- Metni küçük harfe dönüştürme
- Regex ile kelimeleri ayırma
- HashMap kullanarak frekans hesaplama
- Stream API ile sıralama işlemi gerçekleştirme
- En çok kullanılan ilk beş kelimeyi ekrana yazdırma

Java çözümü Python'a göre daha fazla sözdizimi gerektirmiştir. Ancak kullanılan veri yapıları ve nesne yönelimli yaklaşım sayesinde daha düzenli ve sürdürülebilir bir yapı oluşturulmuştur.

Ayrıca Java dili güçlü hata kontrol mekanizmalarına sahiptir. Derleme aşamasında birçok hata tespit edilebildiği için yazılım güvenilirliği artmaktadır. Bu nedenle Java dili büyük ölçekli ve kurumsal yazılım projelerinde yaygın olarak tercih edilmektedir.

Bu çalışmada Java dili kullanılarak geliştirilen çözüm, güçlü veri yapıları sayesinde düzenli ve okunabilir bir yapı sunmuştur.

```

import java.util.*;
import java.util.regex.*;

public class WordFrequency {

    public static void main(String[] args) {

        String text =
            "Bu bir örnek metin. Bu metin içinde bazı kelimeler tekrar ediyor. " +
            "Kelimeler tekrar ettikçe bu kelimeler sayılacak.";

        Map<String, Integer> wordCounts = new HashMap<>();

        // Türkçe karakter desteği
        Matcher matcher = Pattern
            .compile("\\b\\p{L}+\\b")
            .matcher(text.toLowerCase(new Locale("tr", "TR")));

        // Kelimeleri say
        while (matcher.find()) {
            String word = matcher.group();
            wordCounts.put(word,
                wordCounts.getOrDefault(word, 0) + 1);
        }

        // Tüm kelime frekansları
        System.out.println("--- Tüm Kelime Frekansları ---");

        for (Map.Entry<String, Integer> entry : wordCounts.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }
}

```

```

java
28
29
30     for (Map.Entry<String, Integer> entry : wordCounts.entrySet()) {
31         System.out.println(entry.getKey() + ": " + entry.getValue());
32     }
33
34     // İlk 5 kelime
35     System.out.println("\n--- En Çok Geçen İlk 5 Kelime ---");
36
37     wordCounts.entrySet()
38         .stream()
39         .sorted(Map.Entry.<String, Integer>
40             .comparingByValue().reversed())
41         .limit(5)
42         .forEach(entry ->
43             System.out.println(entry.getKey()
44                 + ": "
45                 + entry.getValue()));
46 }

```

Şekil 3,4. Java Kaynak Kodu

Java çözümü Python'a göre daha fazla sözdizimi gerektirmiştir. Ancak kullanılan veri yapıları ve nesne yönelimli yaklaşım sayesinde daha düzenli ve sürdürülebilir bir yapı oluşturulmuştur.

6. C DİLİ İLE GERÇEKLEŞTİRİLEN ÇÖZÜM

C dili ile gerçekleştirilen çözümde işlemler diğer dillere göre daha düşük seviyede gerçekleştirilmiştir. C dilinde hazır veri yapılarının ve gelişmiş standart kütüphanelerin sınırlı olması nedeniyle birçok işlem manuel olarak yapılmıştır. Bu durum geliştirme sürecini daha detaylı hale getirmiş ancak programcıya veri yönetimi üzerinde daha fazla kontrol sağlamıştır.

İlk aşamada metin küçük harfe dönüştürülmüştür. Böylece büyük ve küçük harf farklılıklarının analiz sonucunu etkilemesi engellenmiştir. Daha sonra metin içerisindeki kelimeler strtok()

fonksiyonu yardımıyla ayrılmıştır. Bu fonksiyon belirlenen ayırıcı karakterlere göre metni parçalayarak kelimelerin elde edilmesini sağlamaktadır.

Kelime frekanslarını tutmak amacıyla struct yapısı kullanılmıştır. Bu yapı içerisinde her kelime ve o kelimenin tekrar sayısı saklanmıştır. Yeni bir kelime bulunduğunda mevcut kelimeler ile karşılaştırma yapılmış, daha önce bulunan kelimelerin tekrar sayısı artırılmıştır. Eğer kelime daha önce eklenmemişse yeni bir kayıt oluşturulmuştur.

Frekans hesaplama işlemleri tamamlandıktan sonra kelimeler tekrar sayılarına göre sıralanmıştır. Bu işlem için Bubble Sort algoritması kullanılmıştır. Bubble Sort algoritması basit bir sıralama yöntemi olmasına rağmen küçük veri gruplarında anlaşılır ve uygulanabilir bir çözüm sunmaktadır.

C çözümünde gerçekleştirilen işlemler aşağıdaki gibidir:

- Metni küçük harfe dönüştürme
- strtok() ile kelimeleri ayırma
- Struct kullanarak veri saklama
- Manuel frekans hesaplama
- Bubble Sort ile sıralama işlemi gerçekleştirme
- En çok kullanılan ilk beş kelimeyi ekrana yazdırma

C dili diğer dillere göre daha fazla kod yazılmasını gerektirmiştir. Özellikle string işlemleri, veri saklama işlemleri ve sıralama süreçleri manuel şekilde gerçekleştirildiği için geliştirme süreci daha uzun sürmüştür.

Bununla birlikte C dili düşük seviyeli yapısı sayesinde performans açısından önemli avantajlar sağlamaktadır. Programcı belleği ve veri işlemlerini doğrudan kontrol edebildiği için daha hızlı çalışan uygulamalar geliştirilebilmektedir.

Bu çalışmada C dili kullanılarak geliştirilen çözüm, prosedürel programlama yaklaşımının çalışma mantığını açık şekilde göstermiştir.

C kaynak kodu aşağıda verilmiştir.

```

C basic.c.c X
C basic.c.c > ...
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  #define MAX_WORDS 100
6  #define MAX_WORD_LENGTH 50
7
8  // Kelime ve tekrar sayısını tutan yapı
9  struct Word {
10     char text[MAX_WORD_LENGTH];
11     int count;
12 };
13
14 int main() {
15
16     char text[] =
17         "Bu bir örnek metin. Bu metin içinde bazı kelimeler tekrar ediyor. "
18         "Kelimeler tekrar ettikçe bu kelimeler sayılacak.";
19
20     struct Word words[MAX_WORDS];
21
22     int totalWords = 0;
23
24     // Metni küçük harfe çevir
25     for (int i = 0; text[i] != '\0'; i++) {
26         text[i] = tolower(text[i]);
27     }
28
29     // Kelimeleri ayır
30     char *token = strtok(text, " .,!?");
31
32     while (token != NULL) {

```

```

C basic.c.c X
C basic.c.c > main()
34         int found = 0;
35
36         // Kelime daha önce var mı kontrol et
37         for (int i = 0; i < totalWords; i++) {
38
39             if (strcmp(words[i].text, token) == 0) {
40                 words[i].count++;
41                 found = 1;
42                 break;
43             }
44         }
45
46         // Yeni kelime ekle
47         if (!found) {
48
49             strcpy(words[totalWords].text, token);
50             words[totalWords].count = 1;
51
52             totalWords++;
53         }
54
55         token = strtok(NULL, " .,!?");
56     }
57
58     // Tüm kelime frekanslarını yazdır
59     printf("--- Tüm Kelime Frekansları ---\n");
60
61     for (int i = 0; i < totalWords; i++) {
62         printf("%s : %d\n",
63             words[i].text,
64             words[i].count);
65     }

```

```

basic.c.c x
basic.c.c > main()
66
67 // Basit sıralama
68 for (int i = 0; i < totalWords - 1; i++) {
69     for (int j = i + 1; j < totalWords; j++) {
70         if (words[j].count > words[i].count) {
71             struct Word temp = words[i];
72             words[i] = words[j];
73             words[j] = temp;
74         }
75     }
76 }
77
78 // İlk 5 kelime
79 printf("\n--- En Çok Geçen İlk 5 Kelime ---\n");
80
81 for (int i = 0; i < 5 && i < totalWords; i++) {
82     printf("%s : %d\n",
83           words[i].text,
84           words[i].count);
85 }
86
87 return 0;
88 }

```

Şekil 5,6, 7. C Kaynak Kodu

C dili diğer dillere göre daha uzun ve detaylı kod gerektirmiştir. Özellikle string işlemleri ve sıralama işlemlerinin manuel yapılması geliştirme sürecini zorlaştırmıştır.

Bununla birlikte C dili düşük seviyeli yapısı sayesinde performans açısından önemli avantaj sağlamaktadır.

7. KOD KARŞILAŞTIRMASI

Bu çalışmada aynı problem Python, Java ve C programlama dilleri kullanılarak çözülmüştür. Çözümler incelendiğinde kullanılan programlama paradigmasına bağlı olarak önemli farklılıklar ortaya çıktığı görülmüştür. Her dil aynı problemi çözmesine rağmen kullanılan yöntemler, kod uzunluğu, veri yapıları ve geliştirme süreçleri birbirinden farklı olmuştur.

Python dili en kısa ve en okunabilir çözümü sunmuştur. Bunun temel nedeni Python'un güçlü standart kütüphanelere sahip olması ve sade bir sözdizimi kullanmasıdır. Özellikle Counter veri yapısı sayesinde kelime frekanslarının hesaplanması oldukça kolay hale gelmiştir. Ayrıca regex işlemleri de kısa kodlarla gerçekleştirilebilmiştir. Bu durum geliştirme sürecini hızlandırmış ve kod karmaşıklığını azaltmıştır.

Java dili Python'a göre daha fazla kod yazılmasını gerektirmiştir. Bunun nedeni Java'nın daha detaylı bir sözdizimine sahip olmasıdır. Ancak Java'nın güçlü veri yapıları ve nesne yönelimli yaklaşımı sayesinde daha düzenli ve sürdürülebilir bir çözüm geliştirilmiştir. Özellikle HashMap ve Stream API kullanımı kodun daha kontrollü ve profesyonel görünmesini sağlamıştır.

C dili ise diğer dillere göre en detaylı ve en uzun çözümü gerektirmiştir. Çünkü C dilinde hazır veri yapıları bulunmadığı için birçok işlem manuel şekilde gerçekleştirilmiştir. Kelimelerin saklanması için struct yapısı kullanılmış, frekans hesaplamaları karşılaştırmalar yapılarak

gerçekleştirilmiş ve sıralama işlemleri manuel olarak yazılmıştır. Bu durum geliştirme sürecini daha karmaşık hale getirmiştir.

Kod karşılaştırması aşağıdaki tabloda verilmiştir.

Tablo 2. Programlama Dillerinin Karşılaştırılması

Özellik	Python	Java	C
Kod Uzunluğu	Kısa	Orta	Uzun
Okunabilirlik	Çok Yüksek	Yüksek	Orta
Geliştirme Süresi	Çok Hızlı	Orta	Yavaş
Veri Yapısı Desteği	Güçlü	Güçlü	Sınırlı
Manuel İşlem Miktarı	Az	Orta	Fazla

Tablo incelendiğinde Python dilinin geliştirme kolaylığı açısından önemli avantaj sağladığı görülmektedir. Java dili daha düzenli ve sürdürülebilir bir yapı sunarken, C dili performans avantajı karşılığında daha fazla manuel işlem gerektirmiştir.

Bu sonuçlar programlama dillerinin yalnızca sözdizimi açısından değil, geliştirme süreci, okunabilirlik, bakım kolaylığı ve problem çözme yaklaşımı açısından da birbirlerinden önemli ölçüde farklı olduğunu göstermektedir.

8. ZORLUK ANALİZİ

Bu çalışma sırasında kullanılan Python, Java ve C programlama dilleri geliştirme süreci açısından farklı zorluk seviyeleri sunmuştur. Her dilin sahip olduğu sözdizimi, veri yapıları, kütüphane desteği ve programlama yaklaşımı çözüm sürecini doğrudan etkilemiştir.

Çalışma sırasında en kolay kullanılan dil Python olmuştur. Python dilinin sade ve okunabilir sözdizimi geliştirme sürecini önemli ölçüde kolaylaştırmıştır. Ayrıca güçlü standart kütüphanelere sahip olması sayesinde birçok işlem kısa ve anlaşılır şekilde gerçekleştirilebilmiştir.

Özellikle Counter veri yapısı sayesinde kelime frekanslarının hesaplanması oldukça hızlı bir şekilde yapılmıştır. Regex işlemleri için kullanılan re kütüphanesi de metin içerisindeki kelimelerin kolayca ayrıştırılmasını sağlamıştır. Bu nedenle Python çözümünde daha az kod yazılmış ve geliştirme süreci daha kısa sürmüştür.

Java dili orta seviyede zorluk sunmuştur. Java'nın sözdizimi Python'a göre daha detaylıdır ve daha fazla kod yazılmasını gerektirmektedir. Ancak Java'nın güçlü veri yapıları ve standart kütüphaneleri geliştirme sürecini kolaylaştırmıştır.

Bu çalışmada kullanılan HashMap, Pattern, Matcher ve Stream API yapıları sayesinde kelime frekans hesaplama işlemleri düzenli bir şekilde gerçekleştirilebilmiştir. Bunun yanında Java'nın nesne yönelimli yapısı kodun daha düzenli ve sürdürülebilir olmasını sağlamıştır.

Çalışma sırasında en zor kullanılan dil C dili olmuştur. Bunun temel nedeni C dilinin düşük seviyeli ve manuel işlem gerektiren bir yapıya sahip olmasıdır. Diğer dillere göre daha fazla kontrol sağlamasına rağmen geliştirme süreci daha karmaşık hale gelmiştir.

C dilinde karşılaşılan başlıca zorluklar aşağıdaki gibidir:

- String işlemlerinin manuel yapılması
- Hazır veri yapılarının bulunmaması
- Frekans hesaplama işlemlerinin manuel gerçekleştirilmesi
- Sıralama algoritmasının ayrıca yazılması
- Bellek yönetiminin daha karmaşık olması
- Daha fazla kod yazılması gerekmesi

Özellikle kelimelerin saklanması, karşılaştırılması ve sıralanması işlemleri C dilinde manuel olarak gerçekleştirildiği için geliştirme süreci diğer dillere göre daha uzun sürmüştür.

Genel olarak değerlendirildiğinde Python dili geliştirme kolaylığı açısından en avantajlı dil olurken, C dili en fazla teknik detay ve manuel işlem gerektiren dil olmuştur. Java ise bu iki dil arasında daha dengeli bir yapı sunmuştur.

9. PERFORMANS ANALİZİ

Bu çalışmada kullanılan Python, Java ve C programlama dilleri performans açısından karşılaştırılmıştır. Performans değerlendirmesi yapılırken programların çalışma mantığı, derlenme yapısı, bellek yönetimi ve işlem hızları dikkate alınmıştır.

Performans açısından değerlendirildiğinde en hızlı çalışan dilin C olduğu düşünülmektedir. C dili düşük seviyeli ve derlenen bir programlama dili olduğu için donanımına daha yakın çalışmaktadır. Bu durum işlemci ve bellek kaynaklarının daha verimli kullanılmasını sağlamaktadır.

C dilinde programlar doğrudan makine koduna çevrildiği için çalışma sırasında ek bir yorumlama veya sanal makine katmanı bulunmamaktadır. Ayrıca programcı bellek yönetimi üzerinde doğrudan kontrol sahibi olduğu için gereksiz kaynak kullanımı azaltılabilmektedir. Bu nedenle C dili performans gerektiren uygulamalarda oldukça yaygın şekilde tercih edilmektedir.

Java dili performans açısından C diline yakın sonuçlar sunabilmektedir. Java programları doğrudan makine kodu yerine Java Virtual Machine (JVM) üzerinde çalışmaktadır. İlk bakışta bu durum performans dezavantajı oluşturuyor gibi görünse de Java'nın sahip olduğu JIT (Just-In-Time) optimizasyonları sayesinde çalışma sırasında kodlar optimize edilmektedir.

Bu optimizasyon mekanizmaları sayesinde Java dili özellikle büyük ölçekli uygulamalarda oldukça yüksek performans gösterebilmektedir. Ayrıca Java'nın güçlü bellek yönetimi ve otomatik çöp toplama (Garbage Collection) sistemi program geliştirme sürecini kolaylaştırmaktadır.

Python dili ise yorumlanan bir programlama dili olduğu için diğer dillere göre daha yavaş çalışmaktadır. Python kodları çalışma anında yorumlandığı için işlem süresi artabilmektedir. Ayrıca dinamik veri tipleri ve yüksek seviyeli yapılar performans üzerinde ek yük oluşturabilmektedir.

Bununla birlikte Python dili geliştirme hızının yüksek olması, okunabilirliği ve güçlü kütüphane desteği sayesinde özellikle veri analizi, yapay zeka ve otomasyon gibi alanlarda yaygın şekilde kullanılmaktadır.

Bu çalışma için tahmini performans sıralaması aşağıdaki gibidir:

1. C
2. Java
3. Python

Ancak gerçekleştirilen uygulama küçük ölçekli bir kelime frekans analizi uygulaması olduğu için performans farkı kullanıcı açısından büyük ölçüde hissedilmemektedir. Veri miktarının artması durumunda performans farklarının daha belirgin hale geleceği düşünülmektedir.

10. PARADİGMA YORUMU

Bu çalışma, programlama paradigmaları arasındaki farkların aynı problem üzerinden açık şekilde gözlemlenmesini sağlamıştır. Her programlama dili aynı problemi çözmesine rağmen kullanılan yaklaşım, veri yönetimi yöntemi, kod yapısı ve geliştirme süreci birbirinden farklı olmuştur.

Python dili daha çok hızlı geliştirme ve pratik çözüm üretme yaklaşımına odaklanmaktadır. Yüksek seviyeli yapısı ve güçlü standart kütüphaneleri sayesinde birçok işlem kısa ve anlaşılır kodlarla gerçekleştirilebilmektedir. Bu çalışmada kullanılan Counter ve regex yapıları sayesinde kelime frekans analizi oldukça az kod ile tamamlanmıştır.

Python dilinde programcı çoğu zaman düşük seviyeli detaylarla uğraşmadan doğrudan problemin çözümüne odaklanabilmektedir. Bu durum geliştirme sürecini hızlandırmakta ve kod okunabilirliğini artırmaktadır. Özellikle veri analizi, yapay zeka ve otomasyon gibi alanlarda Python'un yaygın olarak tercih edilmesinin temel nedenlerinden biri de budur.

Java dili ise nesne yönelimli programlama yaklaşımını temel almaktadır. Nesne yönelimli programlama, kodun daha düzenli, modüler ve sürdürülebilir şekilde geliştirilmesini amaçlamaktadır. Java'da veri yapıları ve işlemler belirli bir sistematik içerisinde organize edilmektedir.

Bu çalışmada Java'nın HashMap, Pattern, Matcher ve Stream API yapıları kullanılarak daha düzenli bir çözüm oluşturulmuştur. Java dili Python'a göre daha fazla sözdizimi gerektirse de büyük ölçekli projelerde kod yönetimini kolaylaştırmaktadır.

Java'nın nesne yönelimli yaklaşımı sayesinde kod tekrarının azaltılması, bakım kolaylığı sağlanması ve projelerin daha sürdürülebilir hale gelmesi mümkün olmaktadır. Bu nedenle Java özellikle kurumsal yazılım geliştirme süreçlerinde yaygın olarak tercih edilmektedir.

C dili ise daha düşük seviyeli ve prosedürel programlama yaklaşımını temsil etmektedir. C dilinde programcı veri yönetimi, bellek işlemleri ve sıralama süreçleri üzerinde doğrudan kontrol sahibidir. Bu durum performans avantajı sağlarken aynı zamanda geliştirme sürecini daha karmaşık hale getirmektedir.

Bu çalışmada C dili kullanılarak gerçekleştirilen çözümde string işlemleri, veri saklama işlemleri ve sıralama algoritmaları manuel şekilde geliştirilmiştir.

Programcı birçok işlemi doğrudan kendisi gerçekleştirdiği için kod uzunluğu artmış ve geliştirme süreci daha detaylı hale gelmiştir.

Bununla birlikte C dili donanımına yakın çalıştığı için performans açısından önemli avantajlar sunmaktadır. Bu nedenle işletim sistemleri, gömülü sistemler ve performans gerektiren uygulamalarda yaygın olarak kullanılmaktadır.

Genel olarak değerlendirildiğinde bu çalışma, aynı problemin farklı programlama paradigmaları kullanılarak farklı şekillerde çözülebileceğini göstermiştir. Her dilin kendine özgü avantajları ve dezavantajları bulunmaktadır. Kullanılacak programlama dili; projenin amacı, performans ihtiyacı, geliştirme süresi ve bakım kolaylığı gibi kriterlere göre seçilmelidir.

11. SONUÇ

Bu çalışmada verilen bir metin üzerinde kelime frekans analizi gerçekleştirilmiş ve aynı problem Python, Java ve C programlama dilleri kullanılarak çözülmüştür. Çalışmanın temel amacı farklı programlama dillerinin aynı problem karşısındaki çözüm yaklaşımlarını incelemek ve bu dilleri çeşitli açılardan karşılaştırmaktır.

Gerçekleştirilen uygulamada metin içerisindeki kelimeler ayrıştırılmış, her kelimenin tekrar sayısı hesaplanmış ve en çok kullanılan ilk beş kelime belirlenmiştir. Bu işlemler üç farklı programlama dili kullanılarak gerçekleştirilmiş ve elde edilen sonuçlar karşılaştırılmıştır.

Çalışma sonucunda her programlama dilinin farklı avantajlar sunduğu gözlemlenmiştir.

Python dili kısa ve okunabilir sözdizimi sayesinde geliştirme sürecini önemli ölçüde hızlandırmıştır. Özellikle Counter gibi hazır veri yapıları sayesinde kelime frekans analizi oldukça az kod ile gerçekleştirilebilmiştir. Bu durum Python'un geliştirme kolaylığı ve okunabilirlik açısından önemli avantaj sağladığını göstermektedir.

Java dili ise nesne yönelimli programlama yaklaşımı sayesinde daha düzenli ve sürdürülebilir bir yapı sunmuştur. Kullanılan HashMap, Pattern, Matcher ve Stream API yapıları sayesinde veri işlemleri sistematik şekilde gerçekleştirilmiştir. Java çözümü Python'a göre daha fazla kod gerektirmiş olsa da büyük ölçekli projeler açısından daha kontrollü bir yapı oluşturmuştur.

C dili ise diğer dillere göre daha düşük seviyeli bir yaklaşım sunmuştur. String işlemleri, veri saklama süreçleri ve sıralama işlemleri manuel şekilde gerçekleştirilmiştir. Bu durum geliştirme sürecini daha zor hale getirmiştir. Ancak C dilinin donanımına yakın çalışması performans açısından önemli avantaj sağlamıştır.

Çalışma sırasında elde edilen sonuçlara göre:

- Python dili geliştirme hızı ve okunabilirlik açısından avantaj sağlamıştır.
- Java dili düzenli ve nesne yönelimli yapısıyla güçlü çözümler sunmuştur.
- C dili performans açısından en güçlü sonuçları vermiştir.

Genel değerlendirme sonucunda bu problem için en uygun dilin Python olduğu düşünülmektedir. Çünkü problem daha kısa, anlaşılır ve hızlı geliştirilebilen bir çözüm ile tamamlanmıştır. Özellikle küçük ve orta ölçekli uygulamalarda Python dili geliştirme sürecini önemli ölçüde kolaylaştırmaktadır.

Bununla birlikte performans gerektiren büyük ölçekli sistemlerde C dili tercih edilebilirken, büyük ve kurumsal projelerde Java dili daha uygun bir seçenek olabilir.

Sonuç olarak bu çalışma sayesinde farklı programlama dillerinin aynı problem üzerindeki çözüm yaklaşımları incelenmiş, programlama paradigmaları arasındaki temel farklar değerlendirilmiş ve dillerin avantajları ile dezavantajları karşılaştırılmıştır.